

Initial-Value Problems for Ordinary Differential Equations

ANDREA PROSPERETTI

1 Simple methods for a single first-order ODE

Let us start with a very simple problem

$$\frac{du}{dt} = f(t), \quad u(t=0) = a. \quad (1.1)$$

The solution evidently is

$$u(t) = a + \int_0^t f(\tau) d\tau. \quad (1.2)$$

We want to see how to generate this result numerically. As usual we start by discretizing the independent variable by setting $t^n = n\Delta t$, $t^{n+1} - t^n = \Delta t$ and we write $u(t^n)$, $u(t^{n+1})$ or u^n , u^{n+1} to denote the value of u at time t^n or t^{n+1} .¹ For now we use only one value of Δt . We will see later that it may be necessary to adjust Δt as the integration proceeds.

Suppose we have calculated u up to the time t^n :

$$u(t^n) = a + \int_0^{t^n} f(\tau) d\tau. \quad (1.3)$$

At the later time $t^{n+1} = t^n + \Delta t$ the exact value of u is given by

$$u(t^{n+1}) = u(t^n) + \int_{t^n}^{t^{n+1}} f(\tau) d\tau. \quad (1.4)$$

The problem is then to approximate the integral. Let us consider a few options:

1. We can carry out the integration approximately by keeping f constant and equal to its value at t^n ; in so doing we would find

$$u(t^{n+1}) \simeq u(t^n) + f(t^n)(t^{n+1} - t^n) = u(t^n) + \Delta t f(t^n). \quad (1.5)$$

2. We can carry out the integration approximately by keeping f constant and equal to its value at t^{n+1} ; in so doing we would find

$$u(t^{n+1}) \simeq u(t^n) + f(t^{n+1})(t^{n+1} - t^n) = u(t^n) + \Delta t f(t^{n+1}). \quad (1.6)$$

3. We can carry out the integration approximately by keeping f constant and equal to the average of its values at t^n and t^{n+1} ; in so doing we would find

$$u(t^{n+1}) \simeq u(t^n) + \frac{1}{2} [f(t^{n+1}) + f(t^n)] (t^{n+1} - t^n) = u(t^n) + \frac{\Delta t}{2} [f(t^{n+1}) + f(t^n)]. \quad (1.7)$$

Let us consider the errors affecting these approximations. Near t^n we may write

$$f(t) = f(t^n + (t - t^n)) = f(t^n) + (t - t^n)f'(t^n) + \frac{1}{2}(t - t^n)^2 f''(t^n) + \dots \quad (1.8)$$

¹Note that here n is not an exponent, it is just an index. The convention normally adopted is that spatial indices are written as subscripts, while time indices are written as superscripts. For example, u_j^n means $u(x_j, t^n)$.

With this we have

$$\begin{aligned}\int_{t^n}^{t^{n+1}} f(\tau) d\tau &\simeq \int_{t^n}^{t^{n+1}} \left[f(t^n) + (\tau - t^n) f'(t^n) + \frac{1}{2} (\tau - t^n)^2 f''(t^n) + \dots \right] d\tau \\ &= (t^{n+1} - t^n) f(t^n) + \frac{1}{2} (t^{n+1} - t^n)^2 f'(t^n) + \dots\end{aligned}\quad (1.9)$$

We see therefore that the error affecting (1.5) is proportional to $\Delta t^2 = (t^{n+1} - t^n)^2$. We find the same result for (1.6). To prove it we write

$$f(t) = f(t^{n+1}) + (t - t^{n+1}) f'(t^{n+1}) + \frac{1}{2} (t - t^{n+1})^2 f''(t^{n+1}) + \dots, \quad (1.10)$$

and consider $t < t^{n+1}$ finding

$$\begin{aligned}\int_{t^n}^{t^{n+1}} f(\tau) d\tau &\simeq \int_{t^n}^{t^{n+1}} \left[f(t^{n+1}) + (t - t^{n+1}) f'(t^{n+1}) + \frac{1}{2} (t - t^{n+1})^2 f''(t^{n+1}) + \dots \right] d\tau \\ &= (t^{n+1} - t^n) f(t^{n+1}) - \frac{1}{2} (t^{n+1} - t^n)^2 f'(t^{n+1}) + \dots\end{aligned}\quad (1.11)$$

By adding (1.9) and (1.11) and averaging we find

$$\int_{t^n}^{t^{n+1}} f(\tau) d\tau \simeq \frac{1}{2} [f(t^n) + f(t^{n+1})] - \frac{1}{4} \Delta t^2 [f'(t^{n+1}) - f'(t^n)] + \dots \quad (1.12)$$

But we know that $f'(t^{n+1}) - f'(t^n) \simeq \Delta t f''(t^n)$ so that the previous result can be rewritten as

$$\int_{t^n}^{t^{n+1}} f(\tau) d\tau = \frac{1}{2} [f(t^n) + f(t^{n+1})] - \frac{1}{4} \Delta t^3 f''(t^n) + \dots \quad (1.13)$$

We thus conclude that the error in this case is proportional to Δt^3 rather than Δt^2 so that the *trapezoidal rule* (1.7) is more accurate than the other two.

For the particular case we have considered (1.5) and (1.6) have the same error and seem to be equivalent. There is however an important difference between these two formulae if we consider a more complicated equation than (1.1):

$$\frac{du}{dt} = f(t, u), \quad u(t=0) = a. \quad (1.14)$$

Note that here the unknown is not only present in the derivative, but also in the function f in the right-hand side. The exact solution is now

$$u(t) = a + \int_0^t f(\tau, u(\tau)) d\tau, \quad (1.15)$$

and the analog of (1.4) would then be

$$u(t^{n+1}) = u(t^n) + \int_{t^n}^{t^{n+1}} f(\tau, u(\tau)) d\tau. \quad (1.16)$$

The approximation (1.5) extends very simply:

$$u(t^{n+1}) \simeq u(t^n) + (t^{n+1} - t^n) \Delta t f(t^n, u(t^n)). \quad (1.17)$$

The approximation (1.6) is superficially just as simple

$$u(t^{n+1}) \simeq u(t^n) + \Delta t f(t^n, u(t^{n+1})). \quad (1.18)$$

However, on a closer inspection, we see that, while everything in the right-hand side of (1.17) is known, the unknown value $u(t^{n+1})$ is present in *both* the left- and right-hand sides of (1.18). For this reason, the formula (1.17) is called *explicit* while the formula (1.18) is called *implicit*.

EXAMPLE. Consider the problem

$$\frac{du}{dt} = \beta C e^{-\beta t} u^2, \quad u(0) = a. \quad (1.19)$$

The explicit formula would give

$$u^{n+1} = u^n + \Delta t \beta C e^{-\beta t^n} (u^n)^2, \quad (1.20)$$

but the implicit formula would give

$$u^{n+1} = u^n + \Delta t \beta C e^{-\beta t^{n+1}} (u^{n+1})^2. \quad (1.21)$$

In order to find u^{n+1} we now need to solve the quadratic equation

$$\Delta t \beta C e^{-\beta t^{n+1}} (u^{n+1})^2 - u^{n+1} + u^n = 0. \quad (1.22)$$

This is easily done in this case but if, for example, we had u^3 or u^4 or some other complicated function in place of u we would have to use a numerical method to find the solution such as, for example, the Newton-Raphson method. If we were to apply this method to (1.22) we would proceed as follows. The function F of which we want to find the zero is

$$F(u^{n+1}) = \Delta t \beta C e^{-\beta t^{n+1}} (u^{n+1})^2 - u^{n+1} + u^n \quad (1.23)$$

Every step of the Newton-Raphson iteration requires an index; to avoid having many superscripts and subscripts let us set temporarily $v = u^{n+1}$. Thus we want to find the zero of

$$F(v) = \Delta t \beta C e^{-\beta t^{n+1}} v^2 - v + u^n. \quad (1.24)$$

Applying the Newton-Raphson formula and denoting the iteration index by κ we would find

$$v_{k+1} = v_\kappa - \frac{F(v_\kappa)}{\partial F / \partial v|_{v_\kappa}}, \quad (1.25)$$

or, explicitly,

$$v_{k+1} = v_\kappa - \frac{\Delta t \beta C e^{-\beta t^{n+1}} v_\kappa^2 - v_\kappa + u^n}{2 \Delta t \beta C e^{-\beta t^{n+1}} v_\kappa - 1}. \quad (1.26)$$

The exact solution of (1.19) is easily found by separating the variables

$$\frac{du}{u^2} = \beta C e^{-\beta t} dt. \quad (1.27)$$

Integrate from $t = 0$, where $u = a$, to the generic t , where $u = u(t)$ to find

$$\int_a^{u(t)} \frac{du}{u^2} = \beta C \int_0^t e^{-\beta \tau} d\tau, \quad (1.28)$$

or

$$-\frac{1}{u(t)} + \frac{1}{a} = -C(e^{-\beta t} - 1), \quad (1.29)$$

from which

$$u(t) = \frac{a}{1 + aC(e^{-\beta t} - 1)}. \quad (1.30)$$

It may be noted that, if $aC > 1$, the solution will diverge as t approaches

$$t = -\frac{1}{\beta} \log \left(1 - \frac{1}{aC} \right). \quad (1.31)$$

2 Heun's method

The same difficulty encountered with the implicit method affects the more accurate trapezoidal rule:

$$u^{n+1} = u^n + \frac{1}{2} [f(t^n, u^n) + f(t^{n+1}, u^{n+1})] \Delta t. \quad (2.1)$$

However, the error analysis leading to (1.13) suggests a considerable simplification. Suppose that we can approximate $f(t^{n+1}, u^{n+1})$ with an error of order Δt^2 . Since this term is multiplied by Δt , the final error introduced in the result will be of order Δt^3 , which is the same as the order of the error of the entire formula (2.1) itself. This step therefore would not increase the error of the result, but it might considerably simplify the calculation. We exploit this remark by setting

$$f(t^{n+1}, u^{n+1}) \simeq f\left(t^{n+1}, u^n + \Delta t f(t^n, u^n)\right). \quad (2.2)$$

The error affecting u in the right-hand side is proportional to Δt^2 , but the great advantage is that now f^{n+1} , approximated in this way, is completely known.² This is called *Heun's method* and can be written as a two-step method:

$$\text{Step 1} \quad u^* = u^n + \Delta t f(t^n, u^n), \quad (2.3)$$

$$\text{Step 2} \quad u^{n+1} = u^n + \frac{1}{2} \Delta t [f(t^n, u^n) + f(t^{n+1}, u^*)]. \quad (2.4)$$

The first step is called a *predictor step* for u^{n+1} , the second step is the *corrector step*. This is a simple example of a family of methods called predictor-corrector methods.

There is a whole class of similar methods which can be written as

$$(a) \quad u^* = u^n + \alpha \Delta t f(t^n, u^n), \quad (2.5)$$

$$(b) \quad u^{n+1} = u^n + \Delta t \left[\left(1 - \frac{1}{2\alpha}\right) f(t^n, u^n) + \frac{1}{2\alpha} + f(t^n + \alpha \Delta t, u^*) \right], \quad (2.6)$$

where α is an arbitrary parameter. All these methods have the same accuracy of order Δt^2 . Clearly, by taking $\alpha = 1$, we are back to Heun's method (2.5), (2.6). The choice $\alpha = \frac{1}{2}$ gives the so-called mid-point rule:

$$u^{n+1} = u^n + f\left(\frac{1}{2}(t^n + t^{n+1}), u^n + \frac{1}{2} \Delta t f(t^n, u^n)\right) \Delta t. \quad (2.7)$$

3 Systems of ODEs

The previous methods extend in a pretty straightforward way to systems of ODEs. For example, let us consider how Heun's method would be applied to the system

$$\frac{du_1}{dt} = f_1(t, u_1, u_2), \quad \frac{du_2}{dt} = f_2(t, u_1, u_2). \quad (3.1)$$

The first step is:

$$u_1^* = u_1^n + \Delta t f_1(t^n, u_1^n, u_2^n), \quad (3.2)$$

$$u_2^* = u_2^n + \Delta t f_2(t^n, u_1^n, u_2^n), \quad (3.3)$$

²The proof is as follows. Let us write for brevity $u^* = u^n + f(t^n, u^n) \Delta t$. Then we are interested in $f(t^{n+1}, u^{n+1}) - f(t^{n+1}, u^*)$. We can write

$$f(t^{n+1}, u^*) = f(t^{n+1}, u^* + (u^{n+1} - u^*)) \simeq f(t^{n+1}, u^{n+1}) + (u^{n+1} - u^*) \frac{\partial f}{\partial u},$$

which, since $u^{n+1} - u^*$ is of order Δt^2 , proves that $f(t^{n+1}, u^{n+1}) - f(t^{n+1}, u^*)$ is of order Δt^2 .

and the second step

$$u_1^{n+1} = u_1^n + \frac{\Delta t}{2} [f_1(t^n, u_1^n, u_2^n) + f_1(t^{n+1}, u_1^*, u_2^*)] , \quad (3.4)$$

$$u_2^{n+1} = u_2^n + \frac{\Delta t}{2} [f_2(t^n, u_1^n, u_2^n) + f_2(t^{n+1}, u_1^*, u_2^*)] . \quad (3.5)$$

Although we have only specifically addressed first-order equations and systems, there is no loss of generality because any higher-order equation can be reduced to a system. For example, the equation

$$\frac{d^2 u}{dt^2} + a(t) \frac{du}{dt} + b(t)u(t) = f(t) , \quad (3.6)$$

with initial conditions

$$u(t=0) = u_0 , \quad \left. \frac{du}{dt} \right|_{t=0} = v_0 , \quad (3.7)$$

is evidently equivalent to the two-equations system

$$\frac{du}{dt} = v(t) , \quad (3.8)$$

$$\frac{dv}{dt} = f(t) - a(t)v(t) - b(t)u(t) , \quad (3.9)$$

with initial conditions

$$u(t=0) = u_0 , \quad v(t=0) = v_0 . \quad (3.10)$$

EXAMPLE 1. Let us consider a simple harmonic oscillator with damping; after dividing through by the body's mass the equation of motion is

$$\frac{d^2 x}{dt^2} = -\omega_0^2 x - 2b \frac{dx}{dt} , \quad (3.11)$$

in which ω_0 has the physical meaning of the angular frequency of the oscillator and b is the damping parameter. We associate to this equation the initial conditions

$$x(0) = X_0 , \quad \left. \frac{dx}{dt} \right|_{t=0} = V_0 . \quad (3.12)$$

The first step is to reduce the second-order equation to a system of two first order equations. Evidently we can write this system as

$$\frac{dx}{dt} = v , \quad \frac{dv}{dt} = -\omega_0^2 x - bv . \quad (3.13)$$

Upon comparing with (3.1) we see that

$$x \rightarrow u_1 , \quad u_2 \rightarrow v , \quad f_1(t, u_1, u_2) \rightarrow f_1(t, x, v) = v , \quad f_2(t, u_1, u_2) \rightarrow f_2(t, x, v) = -\omega_0^2 x - bv . \quad (3.14)$$

Then the first step of Heun's method is

$$x^* = x^n + \Delta t v^n , \quad (3.15)$$

$$v^* = v^n + \Delta t [-\omega_0^2 x^n - bv^n] , \quad (3.16)$$

and the second step

$$x^{n+1} = x^n + \frac{\Delta t}{2} [v^n + v^*] , \quad (3.17)$$

$$v^{n+1} = v^n + \frac{\Delta t}{2} [-\omega_0^2 x^n - bv^n - \omega_0^2 x^* - bv^*] . \quad (3.18)$$

The exact solution of the problem (3.11), (3.12) is

$$x(t) = e^{-bt} (A \cos \omega t + B \sin \omega t), \quad \omega = \sqrt{\omega_0^2 - b^2}. \quad (3.19)$$

The constants A and B are found from the initial conditions. For example, $x(0) = A = X_0$, and similaly for B by imposing the second condition in (3.12).

EXAMPLE 2. Let us consider the orbit of a planet around the sun. The orbit is planar and, in its plane, we choose a Cartesian system of coordinates (x, y) with the sun at the origin. After dividing through by the mass of the planet, the equations that govern its motion are

$$\frac{d^2x}{dt^2} = -\frac{GM_s x}{(x^2 + y^2)^{3/2}}, \quad (3.20)$$

$$\frac{d^2y}{dt^2} = -\frac{GM_s y}{(x^2 + y^2)^{3/2}}, \quad (3.21)$$

in which G is the gravitational constant and M the mass of the sun. We decompose this system of equations into four first-order equations:

$$\frac{dx}{dt} = v_x, \quad \frac{dv_x}{dt} = -\frac{GM_s x}{(x^2 + y^2)^{3/2}}, \quad (3.22)$$

$$\frac{dy}{dt} = v_y, \quad \frac{dv_y}{dt} = -\frac{GM_s y}{(x^2 + y^2)^{3/2}}, \quad (3.23)$$

so that

$$f_1(t, x, v_x, y, v_y) = v_x, \quad f_2(t, x, v_x, y, v_y) = -\frac{GM_s x}{(x^2 + y^2)^{3/2}}, \quad (3.24)$$

$$f_3(t, x, v_x, y, v_y) = v_y, \quad f_4(t, x, v_x, y, v_y) = -\frac{GM_s y}{(x^2 + y^2)^{3/2}}. \quad (3.25)$$

Then the first step would be

$$x^* = x^n + \Delta t v_x^n, \quad y^* = y^n + \Delta t v_y^n, \quad (3.26)$$

$$v_x^* = v_x^n - \frac{GM_s \Delta t x^n}{[(x^n)^2 + (y^n)^2]^{3/2}}, \quad v_y^* = v_y^n - \frac{GM_s \Delta t y^n}{[(x^n)^2 + (y^n)^2]^{3/2}}, \quad (3.27)$$

and the second step

$$x^{n+1} = x^n + \frac{1}{2} \Delta t [v_x^n + v_x^*], \quad v_x^{n+1} = v_x^n - \frac{1}{2} GM_s \Delta t \left[\frac{x^n}{[(x^n)^2 + (y^n)^2]^{3/2}} + \frac{x^*}{[(x^*)^2 + (y^*)^2]^{3/2}} \right] \quad (3.28)$$

$$y^{n+1} = y^n + \frac{1}{2} \Delta t [v_y^n + v_y^*], \quad v_y^{n+1} = v_y^n - \frac{1}{2} GM_s \Delta t \left[\frac{y^n}{[(x^n)^2 + (y^n)^2]^{3/2}} + \frac{y^*}{[(x^*)^2 + (y^*)^2]^{3/2}} \right]. \quad (3.29)$$

4 Runge-Kutta methods

The most famous class of methods for the initial-value problem associated with ordinary differential equations (or ordinary differnetial equation systems) goes under the name of *Runge-Kutta methods* of which Heun's method may be considered a simple example. Just as for Heun's method, for each order of accuracy, there is an infinity of variants. The most well known are:

1. *Third-order Runge-Kutta method* (or simply Kutta method) is a three-step method:

$$(a) \quad u^* = u^n + \frac{\Delta t}{2} f(t^n, u^n), \quad (4.1)$$

$$(b) \quad u^{**} = u^n + \Delta t \left[2f\left(t^n + \frac{1}{2}\Delta t, u^*\right) - f(t^n, u^n) \right], \quad (4.2)$$

$$(c) \quad u^{n+1} = u^n + \frac{\Delta t}{6} \left[f(t^n, u^n) + 4f\left(t^n + \frac{1}{2}\Delta t, u^*\right) + f(t^n + \Delta t, u^{**}) \right]. \quad (4.3)$$

The u^* given by the first step is evidently an estimate of u at the midpoint as given by the forward Euler method. The second step can be written as

$$u^{**} = u^n + \Delta t f\left(t^n + \frac{1}{2}\Delta t, u^*\right) + \Delta t \left[f\left(t^n + \frac{1}{2}\Delta t, u^*\right) - f(t^n, u^n) \right]. \quad (4.4)$$

The first two terms are an estimate of u^{n+1} given by the midpoint rule, and the last term is a correction to it. Finally, u^{n+1} is the result of the weighted average of the three slopes, with the central one given more weight than the first and the last on account of its somewhat greater accuracy.

2. The *Fourth-order Runge-Kutta method* is a four-step method:

$$(a) \quad u^* = u^n + \frac{\Delta t}{2} f(t^n, u^n), \quad (4.5)$$

$$(b) \quad u^{**} = u^n + \frac{\Delta t}{2} f\left(t^n + \frac{1}{2}\Delta t, u^*\right), \quad (4.6)$$

$$(c) \quad u^{***} = u^n + \Delta t f(t^n, u^{**}) \quad (4.7)$$

$$(d) \quad u^{n+1} = u^n + \frac{\Delta t}{6} \left[f(t^n, u^n) + 2f\left(t^n + \frac{1}{2}\Delta t, u^*\right) + 2f\left(t^n + \frac{1}{2}\Delta t, u^{**}\right) + f(t^n, u^{***}) \right] \quad (4.8)$$

This is the most famous of the Runge-Kutta methods and it is widely used for the integration of ordinary differential equations; it has an accuracy of order Δt^4 .

5 Adaptive integration

In order to save computation, in solving a specific problem it is desirable to take time steps as long as possible, but not so long as to compromise accuracy. Since, with the methods described, each time step is calculated independently from the preceding ones,³ it is possible to adjust the time step in response to the needs of accuracy and efficiency as the integration proceeds; this process is termed *adaptive integration*.

The key ingredient is to have a way to “sense” how accurate a solution the integration process is producing “in real time,” i.e., as the integration progresses. A standard way to achieve this objective is to take one step of length Δt starting at t^n , and two half-steps of length $\frac{1}{2}\Delta t$, also starting at t^n , and to compare the two solutions thus found, which are two different approximations to u^{n+1} :

$$\Delta = |u_{two\ steps}^{n+1} - u_{one\ step}^{n+1}|. \quad (4.9)$$

In the case of a system of equations we would be looking at the maximum of this difference over all the dependent variables. The general idea is that, if Δ , as computed, is very very small, we can safely increase the time step, for example doubling it for the next step. If, on the other hand, Δ is not sufficiently small, that signals that the time step currently being used is too long and needs to be reduced, e.g. halved.

³For this reason these are called *one-step methods*. There are other classes of methods, called *multi-step methods*, in which taking the next time step involves a few of the preceding ones.

Of course, “large” and “small” must also be related to some scale specific to the problem at hand, so that the quantity to focus on really is not Δ but some normalized form of Δ :

$$\phi = \frac{\Delta_{desired}}{\Delta}, \quad (4.10)$$

where $\Delta_{desired}$ is a measure of the maximum acceptable difference between the two solutions $u_{two\ steps}^{n+1}$ and $u_{one\ step}^{n+1}$. The time step can be increased if ϕ is larger than some limit (e.g., one or a few powers of 10), but it should be decreased if ϕ is smaller than 1. There are several possibilities:

- If, for example, the solution is oscillating with a characteristic amplitude A , one may want to take $\Delta_{desired} = \epsilon A$, with ϵ some small number, e.g., 10^{-6} .
- Another possibility is to use u^{n+1} itself as a scale by taking $\Delta_{desired} = \epsilon |u^{n+1}|$. In this way, the criterion requires that the difference should be a small fraction of the current value of the dependent variable.
- An empirical prescription founded on actual practice is:

$$\Delta t_{new} = \Delta t \phi^\alpha, \quad (4.11)$$

where Δt_{new} is the time step to be used for the next step. Here $\alpha = 0.2$ when $\Delta < \Delta_{desired}$ and the time step is increased, while $\alpha = 0.25$ when $\Delta > \Delta_{desired}$ and the time step is decreased. The quantity to be used for $\Delta_{desired}$ when using this formula is

$$\Delta_{desired} = \epsilon \left(|u| + \Delta t \left| \frac{du}{dt} \right| \right). \quad (4.12)$$